كليـــــــــة العلوم
السملالية - مراكش
FACULTÉ DES SCIENCES
MARRAKECH - SEMLALIA

GRADUATION PROJECT

To obtain the Bachelor degree in Mathematical and Computer Science (MCS) at the Faculty of Sciences Semlalia in Marrakech.

AI-Based Route Planning and Traffic Forecasting for Sustainable Transport

Internship carried out from the 24th of April 2024 until the 29th of June 2024
- Compressed Report -

Defense Scheduled for June 29, 2024

Realized by :

BOUMOULA Abdelouafi

M'KOUKA Btissam

Supervised by :

Pr. Abdellah NABOU, UCA

Jury Members :

Pr. Abdelwahed EL HASSAN, UCA
Pr. Abdellah NABOU, UCA

Session Year : 2023-2024

Acknowledgements

First and foremost, we would like to express our deepest gratitude to ALLAH for granting us the patience, strength, and blessings to reach this significant milestone. The magnitude of our accomplishment would not have been possible without the guidance and support of numerous individuals who played pivotal roles in our journey.

This may be the most challenging part of our writing this report. Not because I am short on people to acknowledge – quite the opposite. Before we began, we want to thank every person who has impact our life indirectly, positively or negatively. Thank you, you contribute to the persons who we are today.

Our heartfelt gratitude goes to our parents, for being the only persons to love us no matter what we do.

We are also grateful to our supervisors and examiners, Dr. Abdellah Nabou and Dr. Abdelwahed El Hassan, for their passion, knowledge, and thorough examination of our research. Their constructive critique, insightful ideas, and rigorous attention to detail have unquestionably improved the depth and rigor of our study. We value their commitment to academic success and their role in molding us into better researchers.We have been undeservedly lucky throughout our project to work with them who are more talented than we are and to get to steal their wisdom and gracefulness and pass it off as our.

My sincere thanks go to our team, this project began as a big, messy thing and required more than just one hand to chisel something comprehensible out of it.

And finally, we are appreciative for the lessons learned, memories formed, and development we have experienced along the way. May our achievements serve as proof of the strength of teamwork, perseverance, and the limitless possibilities that higher education provides.

Abstract

This graduation project focuses on the development and implementation of an AI-based system for detecting emergency vehicles and giving them priority at traffic lights in emergency situations. The primary objective was to create an object detection system for emergency vehicles using deep learning techniques, specifically training the YOLOv8 model on a customized dataset of Moroccan vehicles. The system's accuracy and relevance to local traffic scenarios were ensured through rigorous training and validation processes. In addition, a traffic congestion prediction model was developed using historical traffic data, leveraging Long Short-Term Memory (LSTM) networks conbined with XGBoost model and the accuracy and efficiency were enhanced and reduce prediction error.

The system was successfully simulated using Arduino Uno, sound detectors, and camera for real-time detection and traffic management. Throughout the project, we encountered various challenges, including system integration and data preparation. However, we successfully developed a robust emergency vehicle detection and traffic light priority system. The fulfillment of this project is an initial step toward a more comprehensive and omnipresent traffic management system, with the ultimate goal of creating a safer and more efficient transportation environment by reducing congestion and traffic accidents.

Table of Contents

List of Figures

Chapter 1

General presentation of the project

Whithin this chapter, we will establish the contextual framework of the project and identify the problem. Furthermore, we will not only address the challenges encountered but also provide comprehensive solutions for overcoming these obstacles. Subsequently, we will explore the organization and workflow employed throughout this project.

1.1 Introduction

Vehicular traffic is endlessly increasing everywhere in the world and can cause terrible traffic congestion at intersections. Most of the traffic lights today feature a fixed green light sequence, therefore the green light sequence is determined without taking the presence of the emergency vehicles into account. Therefore, emergency vehicles such as ambulances, police cars, fire engines, etc. must wait in traffic at an intersection which delays their arrival at their destination causing loss of lives and property.In Morocco, especially in urban areas like Marrakesh, the timeliness of ambulance responses is a significant concern. Severe traffic congestion often prevents emergency services from reaching accident sites promptly. In 2022, Morocco recorded 3,499 road deaths, including fatalities from delayed emergency responses due to traffic congestion and logistical issues [4]. Although specific statistics on fatalities solely due to late ambulance responses are not available, it is clear that congestion and infrastructure challenges significantly contribute to delays and adverse outcomes.

In this section, we present an overview of our project that focuses on modelling and simulating a traffic managing by giving emergency vehicles priority in emergency situations.This means that in addition to detecting vehicles, managing traffic lights, we also optimize a model to predict traffic congestion this prediction has led to a growing research area, especially of machine learning of artificial intelligence (AI).

Throughout this report, we will discuss the AI algorithms and techniques we used to make intelligent decisions and automate tasks in the smart cities, using multiple AI algorithms within the same context can often lead to different results because each algorithm has its own strengths, weaknesses, and underlying principles. This comparative analysis allows us to identify which algorithms perform better in terms of accuracy, efficiency, or other relevant metrics and determine which ones yielded the most promising results because the goal is to select the most suitable algorithm or combination of algorithms that can provide improved results, and finally we will also explore the simulation and modeling methods we employed to evaluate the system's performance.

The simulation aspect of the project plays a Key role in testing and refining the AI model. Through simulation, various scenarios and configurations can be evaluated, allowing for the identification of potential issues and the optimization of system performance.

Overall, this project aims to enhance traffic management systems by incorporating advanced algorithms and sensors. Through simulation and modeling, the focus will be on specific functionalities such as detecting ambulances, giving them priority in emergency situations, and predicting traffic congestion to improve overall efficiency and safety on the roads.

1.2 Problematic

Efficient traffic management is critical, especially in urban areas where congestion can significantly impede the movement of emergency vehicles such as ambulances, police and fire truck. Traditional traffic systems face numerous challenges in providing timely and safe passage for these vehicles, often resulting in delayed emergency responses and increased risk to patients. This project aims to address the problematic aspects of current traffic management systems and propose solutions leveraging advanced technologies.

Key issues include :

— Emergency Vehicle Detection : Traditional traffic systems lack the capability to accurately and promptly detect emergency vehicles. This results in inefficient traffic control and delayed priority for ambulances, potentially leading to critical delays in emergency response.

— Dynamic Traffic Light Management : Existing traffic light systems are typically static and do not adapt in real-time to the presence of emergency vehicles. This lack of adaptability can cause significant delays for emergency vehicles, which need the quickest possible route through intersections.

— Traffic Congestion Prediction : Without accurate traffic congestion prediction, it is challenging to manage and optimize traffic flow effectively. This can lead to bottlenecks and increased delays, not only for emergency vehicles but for all road users.

1.3 Challenges and objectives

As beginners in the field of AI research, we encountered a number of difficulties that we overcame in order to finish our project. The first was a lack of information needed to comprehend the task. Despite the fact that it's a recent, complicated project that demands plenty of information, it takes a lot of work to do within the given two months, which is insufficient for such a subject. However, we did enjoy ourselves and gained practical experience using different Python tools and conducting a study.

The primary objective of this project is to develop an intelligent traffic management system that enhances the efficiency and safety of urban transportation by real-time detection and prioritizing emergency vehicles, dynamic traffic light management, traffic congestion prediction, create a predictive model for traffic congestion using historical traffic data and machine learning techniques, such as Long Short-Term Memory (LSTM) networks combined with XGboost additionally, a system integration and simulation to integrate the detection, prioritization, into a cohesive system and simulate its functionality using ArduinoUno, sound detectors to detect siren of emergency situation, and cameras to manage traffic in real-time.

1.4 Planning and tools

The provided Gantt Chart Graph offers a comprehensive overview of the project's timeline. The chart displays a well-structured plan, enabling a clear understanding of the project's progression. Overall, it is a well-structured and comprehensive chart that provides a solid foundation for successful project execution.

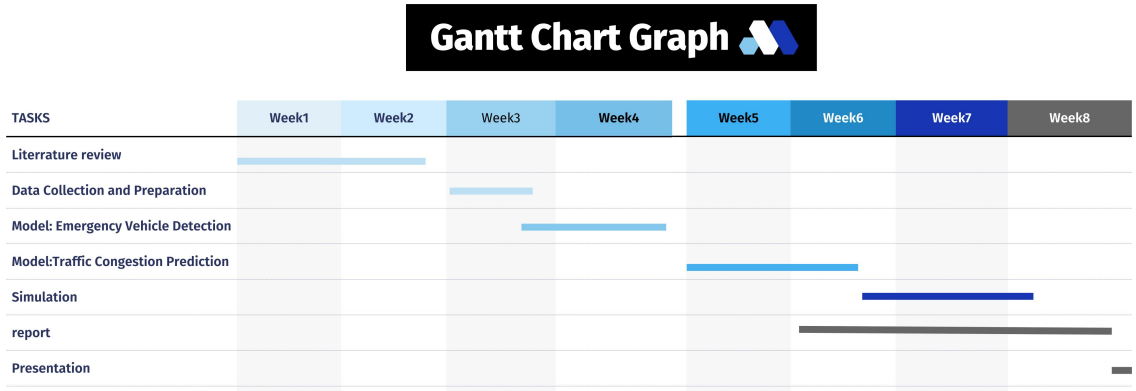


FIGURE 1.1 – Timeline representation

The project's timeline is structured into several critical phases, each presenting its unique challenges and requirements. The initial phase involves an extensive Literature Review, necessitating thorough exploration on platforms like Google Scholar to grasp the intricacies of the topic and identify gaps for proposing innovative solutions. Following this, the Data Collection and Preparation phase emerges as a significant time investment, particularly challenging for obtaining precise, country-specific data, especially concerning emergency vehicle patterns, which are sporadic and rare. This phase is crucial as it lays the foundation for robust model development.

Subsequently, the focus shifts to Model Development, starting with Emergency Vehicle Detection. This stage demands meticulous algorithm selection and parameter tuning to ensure accurate and reliable detection under varied conditions. Simultaneously, developing models for predicting Traffic Congestion requires sophisticated data analysis techniques and modeling approaches to optimize urban mobility solutions effectively. Each phase requires dedicated effort and attention to detail to achieve comprehensive research outcomes and ensure the project's success.

Chapter 2

Technologies and Approaches for Intelligent Traffic Management Systems

Efficient traffic control is essential for smooth driving conditions and pollution reduction. In addition emergency vehicles such as ambulances, fire trucks, and police, provide emergency response to help save lives and ensure security in our society. During emergency conditions, the response time of the emergency vehicle is of significant importance and, in some cases, a few seconds of time saving may save lives. For example, in the case of an ambulance, each minute of delay can reduce the survival rate of patients with cardiac arrest by 7-10 per cent [3]. This project aims to develop an intelligent traffic control system that detects emergency vehicles, their situation (Emergency or not) and predicts traffic congestion. The system uses image processing algorithms to prioritize emergency vehicles by switching traffic lights to green and sensor detector to analyze siren. By integrating these technologies, the system enhances traffic flow, reduces congestion, and ensures the efficient passage of emergency vehicles, significantly improving urban traffic management and public safety.

2.1 Implemented Technologies

2.1.1 Machine learning

In this project, machine learning plays a key role in analyzing traffic patterns, predicting congestion, detecting emergency vehicles, and enhancing image processing algorithms.

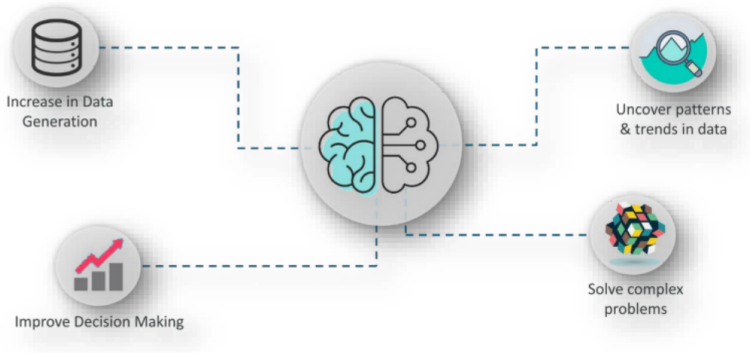


FIGURE 2.1 – The critical points in machine learning

2.1.1.1 Recurrent Neural Network(RNN)

Recurrent Neural Network (RNN) is a type of neural network where the output from the previous step is fed as input to the current step. In traditional neural networks, all the inputs and outputs are independent of each other. However, in cases where it is required to predict the next word of a sentence, the previous words are required and hence there is a need to remember them.

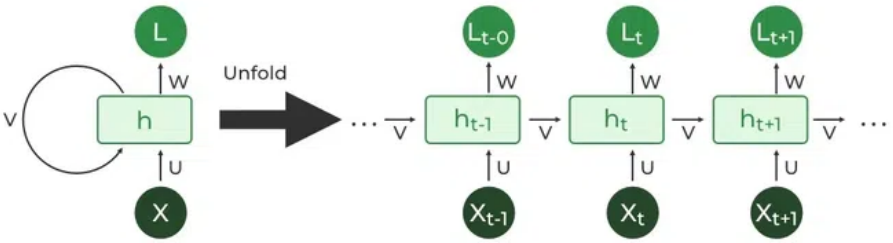


FIGURE 2.2 – Illustration of a recurrent neural network [1]

2.1.1.2 Long Short-Term Memory LSTM

A traditional RNN has a single hidden state that is passed through time, which can make it difficult for the network to learn long-term dependencies. LSTMs address this problem by introducing a memory cell, which is a container that can hold information for an extended period. LSTM networks are capable of learning long-term dependencies in sequential data. LSTMs can also be used in combination with other neural network architectures, and XGBOOST for prediction.

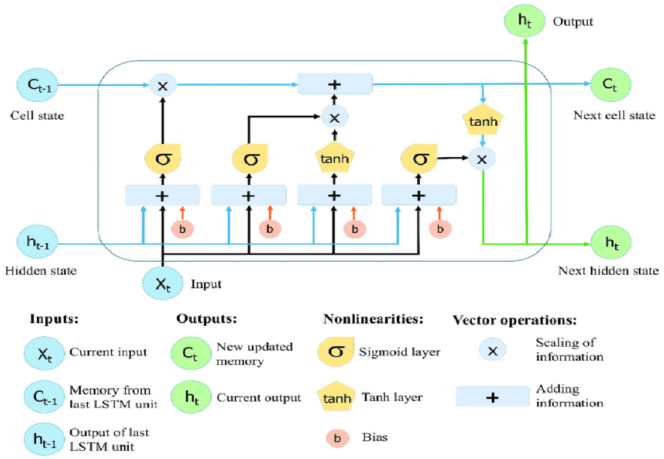


FIGURE 2.3 – Illustration of a recurrent neural network [2]

2.1.2 Image Processing

Before we start to process images, it's important for us to establish the dimensions of the image. The number of pixels does not always determine the size of the picture's height and width. On a digital computer, image processing is required for the advancement of image management. Over the last few years, they've grown at an incredible rate. There are a range of applications, from healthcare and entertainment to historic studies in the field of geographical sciences, for example, remote sensing.
The management of digital images, based on today's data culture, is a pillar of the Mixed Media System. Advanced signal processing techniques, together with picture-specific technologies, are part of digital image processing. The image as an example is a visual representation of something whereas a digital image is a binary representation of visual data.

•Image Enhancement :
The use of visual enhancement to enhance a photograph's image quality or to emphasize certain elements Examples are the enhancement of contrast, brightness adjustment, brightening, and noise reduction.

•Image Restoration :
Restoration of images is a method to prevent or reduce image degradation caused by capturing, transmission, and capacity limitation ; other activities include motion blur, defocus, noise reduction, and damaged or bad photo correction.

•Image Analysis :
The process of analyzing images involves exploiting techniques like segmentation, object recognition and tracking, feature extraction, pattern recognition, and image classification so that valuable information can be obtained from the pictures. Computer vision, healthcare imaging, and remote sensing are among the fields that use this significant technology.

2.1.3 Computer Vision

. By enabling machines to "see" and comprehend visual data, computer vision opens up possibilities for tasks that require visual perception, enabling machines to interact with and understand the world in a more human-like way. For example, emergency vehicles detection by camera involves utilizing computer vision algorithms to analyze video feeds or images captured in order.

2.1.3.1 YOLO

From a machine learning perspective, YOLO utilizes deep learning techniques, specifically convolutional neural networks (CNNs), to learn and detect objects in images and videos. It uses a single neural network architecture to directly predict bounding boxes and class probabilities for multiple objects in a single pass, hence the name "You Only Look Once." YOLO's training process involves optimizing the network's parameters based on annotated data, which is a fundamental aspect of machine learning.

YOLO has achieved remarkable performance in the context of detecting emergency vehicles, delivering excellent results by analyzing the live video feed from security cameras yolo can quickly identify and show accuracy.

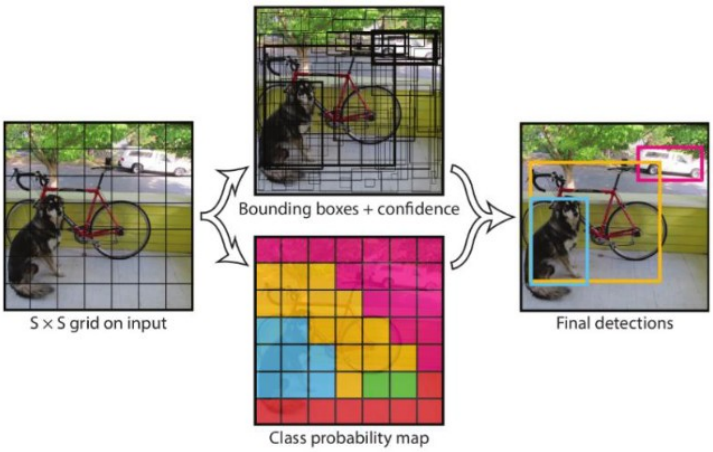


FIGURE 2.4 – YOLO object detection process [3]

The Importance of YOLO :

Other object detection methods, like regionalized neural networks (R-CNN), including other variants like fast R-CNN, focus on a specific region of the image and train each component separately. This process requires the R-CNN to classify 2000 regions per image, which makes it very time-consuming (47 seconds per individual test image). It therefore cannot be implemented in real time. This makes object detection networks such as R-CNN harder to optimize and slower than YOLO. The YOLO model is much faster (45 frames per second) and easier to optimize than previous algorithms, because it is based on an algorithm that uses only a single neural network to execute all elements of the task [5].

The model	Frames Per Second (FPS)	Real-time speed
Fast YOLO	155	YES
YOLO	45	YES
YOLO VGG-16	21	NO
FAST R-CNN	0.5	NO
FAST R-CNN VGG-16	7	NO
FAST R-CNN ZF	18	NO

FIGURE 2.5 – comparison between R-CNN and YOLO

Why Should We Use YOLOv8 ?

Here are a few main reasons why we should consider using YOLOv8 for next computer vision project :

1. YOLOv8 has a high rate of accuracy measured by Microsoft COCO and Roboflow 100.

2. YOLOv8 comes with a lot of developer-convenience features, from an easy-to-use CLI to a well-structured Python package.

3. There is a large community around YOLO and a growing community around the YOLOv8 model, meaning there are many people in computer vision circles who may be able to assist you when you need guidance.

YOLOv8 achieves strong accuracy on COCO. For example, the YOLOv8m model (the medium model) achieves a 50.2% mAP when measured on COCO. When evaluated against Roboflow 100, a dataset that specifically evaluates model performance on various task-specific domains, YOLOv8 scored substantially better than YOLOv5. Furthermore, the developer-convenience features in YOLOv8 are significant. As opposed to other models where tasks are split across many different Python files that you can execute, YOLOv8 comes with a CLI that makes training a model more intuitive. This is in addition to a Python package that provides a more seamless coding experience than prior models [6].

2.2 Literature Review

In the literature review, among seven several scientific articles were reviewed within the same general context of our project. These articles highlighted the importance of employing advanced techniques in image processing, machine learning, and predictive modeling to address traffic management challenges effectively. By studying these articles, we gained a deeper understanding of existing solutions and were able to select those that best suited our needs. However, it is not easy to implement all the functionalities at once Among the various approaches explored, two key ideas emerged as potential avenues for improving accuracy in our project.

The first idea stemmed from the recent advancements in object detection algorithms, particularly the latest version(v8) of You Only Look Once (YOLO). YOLO has demonstrated remarkable performance in real-time object detection tasks, making it a promising candidate for accurately identifying emergency vehicles in traffic scenes. By leveraging the capabilities of YOLO, we aim to enhance the precision and efficiency of our vehicle detection system, thereby improving overall traffic control.

The second idea revolves around the utilization of Long Short-Term Memory (LSTM) networks for congestion prediction, combined with a boosting model XGBoost to reduce the error values. LSTM networks excel in capturing temporal dependencies in sequential data, making them well-suited for forecasting future traffic conditions based on historical data. By integrating LSTM with a boosting model, such as XGBoost, we aim to exploit the complementary strengths of both approaches to achieve more accurate predictions of congestion patterns. This hybrid approach has the potential to enhance the robustness and accuracy of our traffic prediction system, ultimately leading to more effective traffic management strategies.

In addition to exploring advanced machine learning techniques, another promising avenue identified in the literature involves the integration of simulation environments with physical hardware, such as ArduinoUNO ,

Arduino boards, camera, sound detectors and LEDs ...

2.3 Related Work

During our analysis of various scientific articles, we selected a few for their pertinence, especially those concerning object detection techniques using Image Processing, which reviews several approaches for intelligent traffic control system with prioritization during the detection and approving the emergency situation.

In the article titled "A Transfer-Learning-Based Approach for Emergency Vehicle Detection" [3], author Abubakar M. Ashir, Department of Computer Engineering, Faculty of Engineering, Tishk International University, Erbil,Iraq conducted a study on computer-vision based approach for real-time detection of different types of emergency vehicles under heavy traffic conditions it proposed a method of automatic detection of four different categories of emergency vehicle irrespective of the vehicle's shapes, models ormanufacturers using modified version of YOLOv5 object detection algorithm. The proposed model developed here is based on 4 classes which are (Firetrucks, Ambulance, Police Car, and Normal Cars) classes. The top layers (fully connected layers) of the YOLO algorithm were re-designed and retrained to get new learned weights while freezing the bottom layers (convolutional layers) and retaining the pre-trained YOLOv5 weights.

The second interested article titled "LSTM network : a deep learning approach forshort-term traffic" [7] authors Zheng Zhao1, Weihai Chen1 , Xingming Wu1, Peter C. Y. Chen2, Jingmeng Liu11School of Automation Science and Electrical Engineering, Beihang University, Beijing, People's Republic of China2Department of Mechanical Engineering, National University of Singapore .Experiments are conducted to validate the efficiency of the proposed forecast model. According to the comparison between 5 models , the performance of the proposed LSTM network is better than SAE, RBF, SVM and ARIMA model,especially when the forecast time is long.The study focuses on traffic volume prediction, but a comprehensive traffic forecast which includes travel time, traffic speed and occupancy has more significance for commuters.

After reviewing these articles, we were inspired to enhance the existing work to achieve higher accuracy and improved performance using the latest versions.Inspired by those articles we decided to implement and refine the YOLOv8 algorithm for more efficient and accurate detection of emergency vehicles and to improve traffic volume prediction by combining LSTM with advanced boosting techniques like XGBoost. By integrating these modern methodologies and leveraging the latest versions of these models, we aspire to develop a more robust and precise intelligent traffic control system.

Chapter 3

Intelligent Traffic Management Systems

In the previous chapter, we outlined the design of the emergency vehicle detection model and the traffic congestion prediction model, elucidating the various stages of development in detail.

In this chapter, we will introduce the working environment, programming language, and tools utilized to execute our projects. Subsequently, we will delve into the obtained results and conduct a comparative analysis.

3.1 Detection of Emergency Vehicles

3.1.1 Model Testing

Once the training phase is completed, we use the model to make predictions on test images. This script is designed to employ a pre-trained model to make predictions or detect objects in new data. Specifically, it loads the trained model, processes the test images, and outputs the detected objects along with their corresponding confidence scores and bounding boxes. This allows us to evaluate the model's performance on unseen data and verify its accuracy and robustness in real-world scenarios. The results can then be visualized and analyzed to further refine and improve the model if necessary.

Here are some tested images :

FIGURE 3.1 – Validation Batch for Emergency Vehicle Detection

And also, We tested the model on a video of a private ambulance navigating the streets of Marrakech. The results were impressive, with the model achieving high accuracy rates of approximately 92% to 95%. This demonstrates the model's effectiveness in real-world scenarios, accurately detecting emergency vehicles even in complex urban environments. The high accuracy rates validate the robustness and reliability of our model in identifying emergency vehicles, contributing to its potential utility in intelligent traffic control systems.

3.1.2 Model Training and Performance Analysis

The model demonstrates effective learning as evidenced by the decreasing training loss, indicating its improving capability in identifying objects within the training dataset. Moreover, the increasing precision and recall observed on the validation set imply enhancements in the model's proficiency at accurately detecting objects in previously unseen images. Additionally, rising mean Average Precision (mAP) scores further signify an overall enhancement in the model's object detection performance. After the training we can visualize the result in the graphs below :

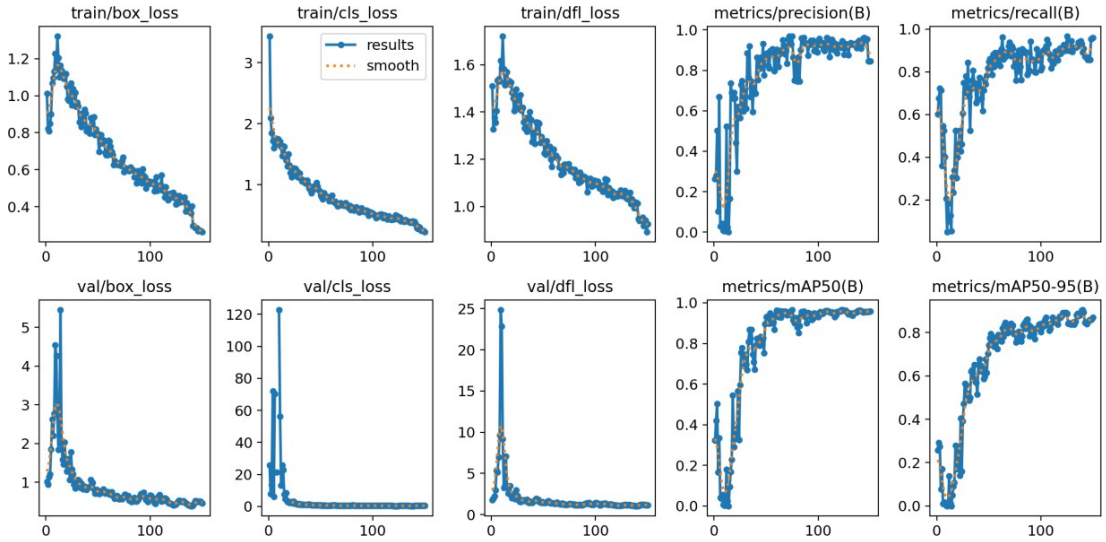


FIGURE 3.2 – Training results performance evaluation

The train/box_loss, train/cls_loss, and train/dfl_loss show a continuous decrease over time, indicating that the model is effectively learning to localize objects, classify them accurately, and refine the bounding box predictions. This demonstrates a robust training process.However, it is worth noting that these loss components don't reach a plateau or stagnation, which suggests that further training could potentially lead to even better performance.On the other hand, the other metrics, such as validation loss exhibit positive signs. The validation loss remains low, indicating that the model is performing well on unseen data.Overall,these graphs indicate that the YOLO model is successfully detecting emergency vehicles with good precision and recall.

3.1.3 Model Evaluation and Results

3.1.3.1 Confusion Matrix

The confusion matrix analysis of the YOLO model for detecting emergency vehicles reveals a nuanced performance across different categories. As seen in following figure the accuracy metrics indicate strong performance, with fire trucks achieving a perfect detection rate of 100%, closely followed by ambulances at 95%, and police vehicles at 83%.

However, the model exhibits notable misclassification patterns : 11% of ambulances are mistakenly identified as police vehicles, and 25% as background objects. Similarly, 5% of police vehicles are misclassified as ambulances, with 6% classified as background.

These findings highlight the model's strengths in accurately detecting fire trucks and maintaining high accuracy for ambulances and police vehicles, yet weaknesses persist in distinguishing between ambulances and police cars, as well as in the tendency to classify them as background objects. Further, false positives are prevalent, notably with 75% of background objects being misclassified as police vehicles. Overall, while the model demonstrates strong capabilities for certain vehicle types, improvements are needed to enhance the differentiation between similar classes and reduce false positives in complex scenarios.

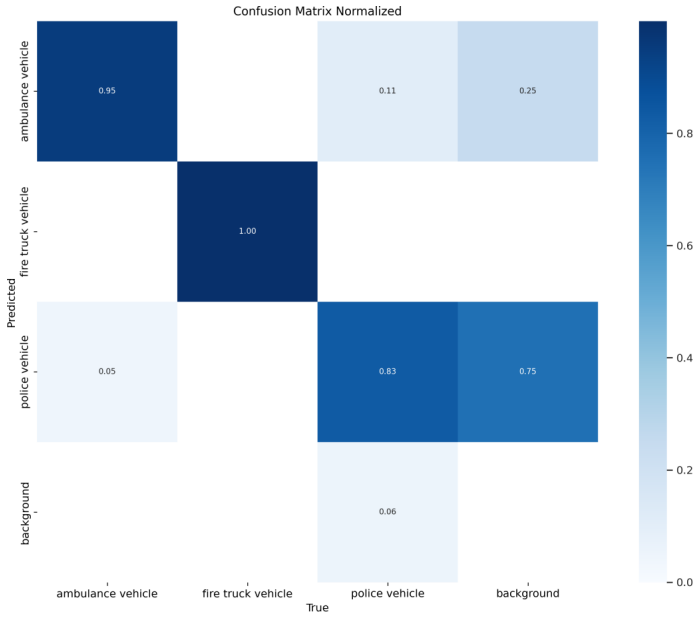


FIGURE 3.3 – Confusion Matrix Normalized

- **Conslusion :**

In conclusion, our YOLO model demonstrates robust performance in detecting emergency vehicles, achieving high accuracy rates for fire trucks, ambulances, and police vehicles.

3.1.4 Real Experimentation

After evaluating the YOLOv8 model and acknowledging its exceptional performance, which yielded the most favorable results, we proceeded to implement the model using the following Python code :

```python
def update_frame():
    global audio_detected
    ret, frame = cap.read()
    frame = cv2.resize(frame, (640, 480))
    result = model(frame, stream=True)
    visual_detected = False

    for info in result:
        boxes = info.boxes
        for box in boxes:
            confidence = box.conf[0]
            confidence = math.ceil(confidence * 100)
            Class = int(box.cls[0])
            if confidence > 60:
                x1, y1, x2, y2 = box.xyxy[0]
                x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2)
                cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 0, 255), 5)
                cvzone.putTextRect(frame, f'{classnames[Class]} {confidence}%', [x1 + 8, y1 + 100], scale=1.5, thickness=2)
                if Class in [0, 1, 2]:
                    print("Emergency vehicle detected  !")
                    visual_detected = True

    print(f"audio_detected: {audio_detected}, visual_detected: {visual_detected}")

    if visual_detected and audio_detected:
        print("Emergency vehicle with siren detected ! Sending the signal to the Arduino...")
        arduino.write(b'1')
        time.sleep(0.1)
        visual_detected = False
        audio_detected = False
    else:
        arduino.write(b'0')

    cv2image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGBA)
    img = Image.fromarray(cv2image)
    imgtk = ImageTk.PhotoImage(image=img)
    label.imgtk = imgtk
    label.configure(image=imgtk)
    label.after(10, update_frame)
```

FIGURE 3.4 – Code of Detection by Intern Camera

Here is the subsequent result :

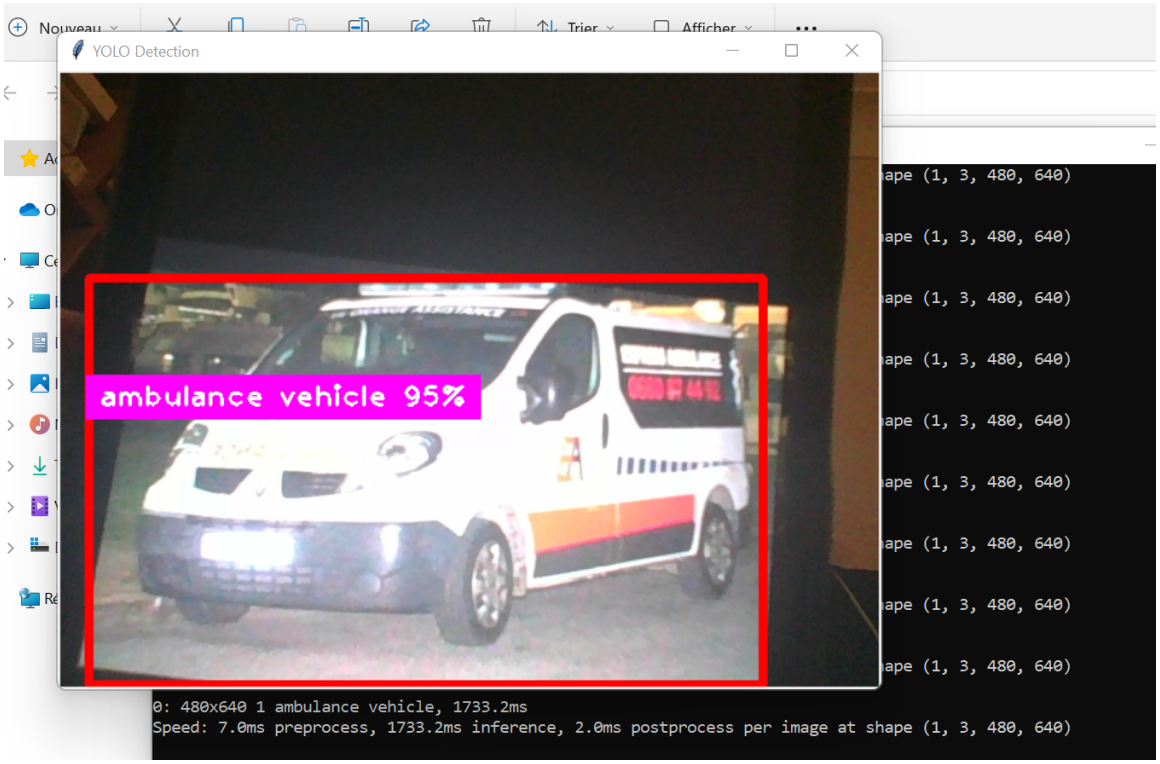


FIGURE 3.5 – Results of Detection using intern Camera

It demonstrates the successful real-time detection of an ambulance vehicle using computer vision technology. The system has identified the ambulance with a high confidence level of 95%, as indicated by the label overlay. This kind of rapid and accurate vehicle detection is crucial for intelligent traffic management systems, especially for prioritizing emergency vehicles.

3.2 Traffic congestion prediction

3.2.1 Software Tools - Libraries

In our programs, we used a set of libraries to leverage predefined functions. They are called at the beginning of the program.

— NumPy (np) : is a powerful Python library used for numerical computations on arrays and matrices. It provides functionalities for efficient data manipulation and mathematical operations.

— Pandas (pd) : is a Python library specialized in data manipulation and analysis. It offers powerful data structures, notably DataFrames, for easy handling of tabular data.

— Plotly press (px) and Plotly Graph Objects (go) : Plotly is an interactive data visualization library in Python. Plotly Express provides a user-friendly interface for quickly creating plots, while Plotly Graph Objects offer more precise control over creating custom graphics.

— Scikit-learn (sklearn) : is an open-source Python library that provides simple and efficient tools for machine learning and data exploration. It offers various machine learning algorithms, data preprocessing tools, and utilities for model evaluation.

— TensorFlow (tensorflow) : widely used for creating, training, and deploying deep learning models. TensorFlow offers flexibility, scalability, and high performance for building complex neural networks.

3.2.2 Basic model LSTM

3.2.2.1 Model definition

In this part, a basic LSTM model is defined using the Keras Sequential API. The model consists of an LSTM layer with 50 units as the primary layer, followed by a Dense layer with a single unit for output. The model is compiled with the Adam optimizer and Mean Squared Error (MSE) loss function.

```
#Model Definition
model_base = Sequential()
model_base.add(LSTM(50, input_shape=(1, X_reshaped.shape[2])))
model_base.add(Dense(1))
```

FIGURE 3.6 – Definition of our basic model LSTM

3.2.2.2 Model training and Prediction

Here, the model is trained using the fit() method. The training data (X_train and y_train) are used for training. The training process is configured to run for 20 epochs with a batch size of 32. Validation data (X_test and y_test) are provided to evaluate the model's performance during training. After training, the model is used to make predictions on the test data (X_test).

```
# Model training
history_base = model_base.fit(X_train, y_train, epochs=20, batch_size=32, validation_data=(X_test, y_test), verbose=2)

# Model Prediction
predictions_base = model_base.predict(X_test)
rmse_base = sqrt(mean_squared_error(y_test, predictions_base))
```

FIGURE 3.7 – Code of our model training and prediction

3.2.2.3 Model Testing

In the context of machine learning, testing a model involves using it to make predictions on a set of data that it has not seen before. This is crucial for evaluating the model's performance and generalization ability.

The model makes predictions on the test data (X_test) using the predict method. To visualize the performance of our basic LSTM model, we utilized Plotly to create interactive plots.

First, we created a line plot to compare the real and predicted traffic situations, allowing us to visually assess how well the model's predictions align with the actual values. We used the fig.update_layout method to set the title to "Comparison of Real and Predicted Values (Basic Model)" and labeled the x-axis as "Samples" and the y-axis as "Traffic Situation".

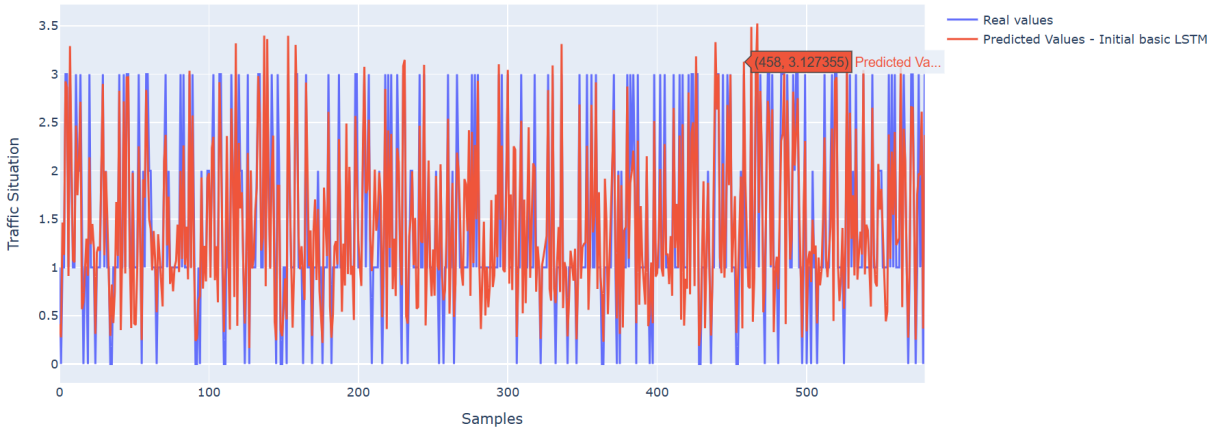


FIGURE 3.8 – Basic Model : Comparison of Real and Predicted Values

Here, a comparison is made between the actual traffic situation values (y_test) and the predicted values (predictions_base) generated by our basic LSTM model. Initially, a pandas DataFrame named comparison_df is created with two columns : 'Real Values' for the true labels and 'Predicted Values' for the model's predictions. The flatten() method is used to convert both arrays into 1D arrays to ensure they can be effectively integrated into the DataFrame. Following this, a random sample of 10 rows is selected from comparison_df using the sample(10) method.

	real values	Predicted Values
570	3.0	2.624698
329	1.0	0.726044
84	1.0	1.079564
78	1.0	1.175381
416	1.0	0.534436
184	1.0	1.105114
501	1.0	1.186373
66	1.0	1.204182
307	1.0	0.787683
72	1.0	1.120828

FIGURE 3.9 – Basic Model : Real Values and Predicted Values

3.2.2.4 Model Evaluation

Model evaluation is a critical step in the machine learning pipeline, providing insights into how well a model performs on unseen data. The evaluation process typically involves comparing the model's predictions to the actual values and computing various metrics to quantify the model's accuracy and reliability. In this example, we use several key metrics : Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE). These metrics provide a comprehensive view of the model's performance.

Additionally, we calculated key evaluation metrics, including Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE). We visualized these metrics using a bar chart to provide a clear view of the model's performance. The fig_metrics.update_layout method was employed to title this plot "Evaluation Metrics" and label the x-axis as "Metric" and the y-axis as "Value". This plot was saved as "base_metrics_evaluation.html". These visualizations and metrics provide a comprehensive overview of the model's accuracy and reliability, facilitating an in-depth evaluation of its predictive capabilities.

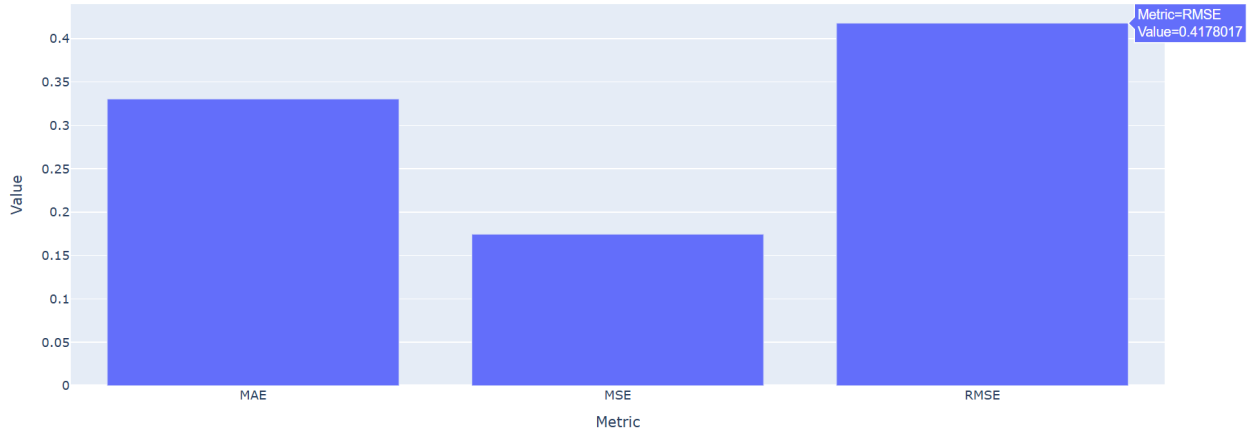


FIGURE 3.10 – Basic Model Evaluation based on RMSE, MAE, MSE values

The results of the Basic LSTM model show an RMSE of 0.397, an MAE of 0.31, and an MSE of 0.15. These values indicate that the model has a significant margin of error in its predictions. To improve the model further, we could consider optimizing hyperparameters such as the number of layers, units per layer, and dropout rates.

3.2.3 Improved LSTM Model

3.2.3.1 Model definition

In this section, we define an improved LSTM model using the Keras Sequential API. The model architecture includes three LSTM layers, each followed by a Dropout layer to reduce overfitting. The first two LSTM layers have 100 units, while the third LSTM layer has 50 units. The Dropout layers have a dropout rate of 30%. A final Dense layer with a single unit is used for output. The model is compiled with the Adam optimizer and the Mean Squared Error (MSE) loss function.

```
# Improved LSTM model definition
model_improved = Sequential()
model_improved.add(LSTM(100, return_sequences=True, input_shape=(1, X_reshaped.shape[2])))
model_improved.add(Dropout(0.3))
model_improved.add(LSTM(100, return_sequences=True))
model_improved.add(Dropout(0.3))
model_improved.add(LSTM(50))
model_improved.add(Dropout(0.3))
model_improved.add(Dense(1))

model_improved.compile(optimizer='adam', loss='mean_squared_error')
model_improved.summary()
```

FIGURE 3.11 – Definition of our improved model LSTM

3.2.3.2 Model training and Prediction

The improved model is trained using the fit() method. Training data (X_train and y_train) are used, with the process configured to run for 100 epochs and a batch size of 64. Validation data (X_test and y_test) are used to evaluate the model's performance during training. After training, the model makes predictions on the test data (X_test). The Root Mean Squared Error (RMSE) is calculated to evaluate the model's performance.

```python
# model training
history_improved = model_improved.fit(X_train, y_train, epochs=100, batch_size=64, validation_data=(X_test, y_test), verbose=2)

# model prediction
predictions_improved = model_improved.predict(X_test)
rmse_improved = sqrt(mean_squared_error(y_test, predictions_improved))
```

FIGURE 3.12 – Code of our improved model training and prediction

3.2.3.3 Model Testing

A line plot to compare the real and predicted traffic situations by our improved model.

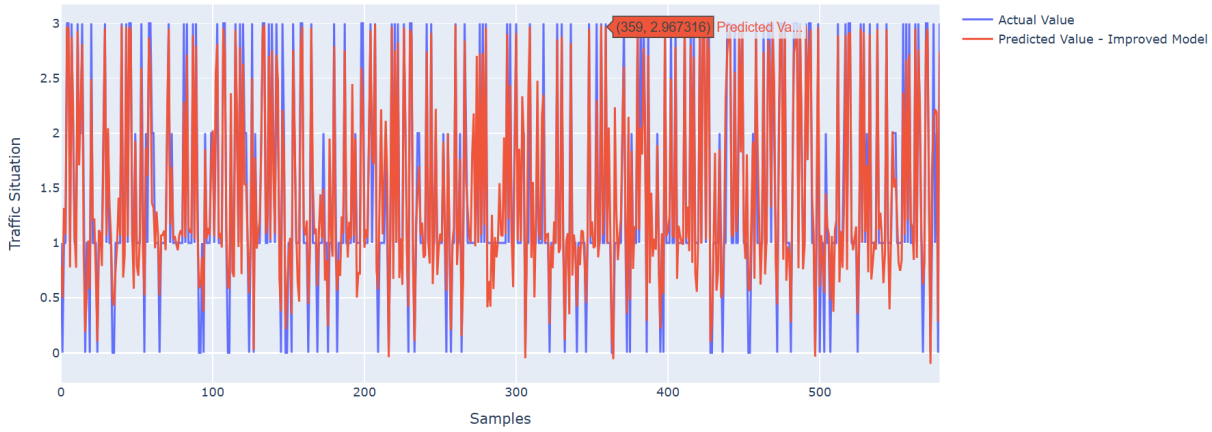


FIGURE 3.13 – Improved model : Comparison of Real and Predicted Values

Here, a comparison between the actual traffic situation values and the predicted values generated by our improved LSTM model.

	Actual Value	Predicted Value
77	1.0	0.901716
91	0.0	0.560345
533	3.0	2.906408
80	1.0	1.050363
345	1.0	0.759833
569	1.0	0.761129
194	2.0	1.831914
292	1.0	1.812472
218	3.0	3.023385
104	1.0	1.033510

FIGURE 3.14 – Improved model : Real Values and Predicted Values

3.2.3.4 Model Evaluation

Here too, we calculated key evaluation metrics, including Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE).

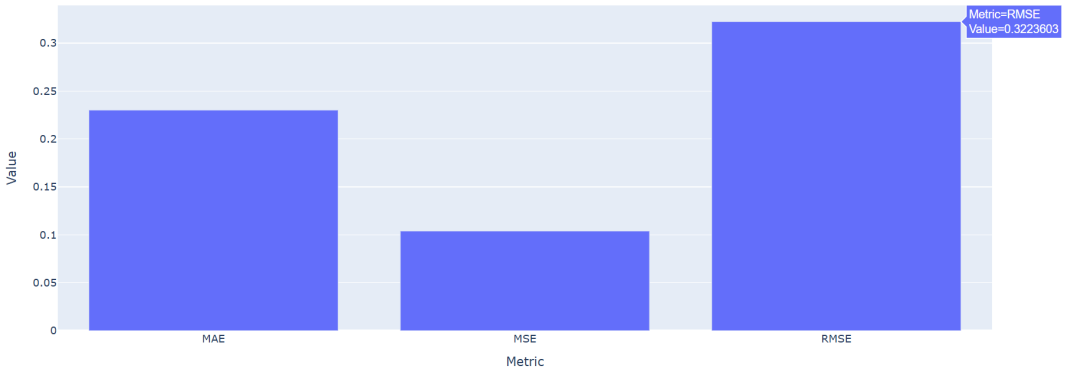


FIGURE 3.15 – Improved model : Basic Model Evaluation based on RMSE, MAE, MSE values

The Improved LSTM model shows a significant enhancement over the Basic LSTM model it demonstrates a notable reduction in prediction errors. This approve that optimizing hyperparameters, adjusting the network architecture, and adding dropout layers—have positively impacted the model's performance. The lower error metrics indicate more accurate and reliable predictions, showcasing the effectiveness of the enhancements applied to the LSTM model.

Model	**RMSE**	**MAE**	**MSE**
Basic LSTM	0.397	0.31	0.15
Improved LSTM	0.316	0.22	0.1

TABLE 3.1 – Comparison of Basic and Improved LSTM Models Performance

3.2.4 Combin LSTM Model with XGBoost

3.2.4.1 Model definition

The implemented script leverages various machine learning techniques for traffic prediction and management. It combines LSTM (Long Short-Term Memory) neural networks and XGBoost models to forecast traffic situations based on time, date, and vehicle counts. The LSTM architecture is designed with multiple layers and dropout regularization for enhanced prediction accuracy. The first two LSTM layers have 100 units, while the third LSTM layer has 50 units. The Dropout layers have a dropout rate of 30%. A final Dense layer with a single unit is used for output. The model is compiled with the Adam optimizer and the Mean Squared Error (MSE) loss function.

3.2.4.2 Model training and Prediction

The model training and prediction process involves the strategic design of an LSTM network, followed by combining its output with an XGBoost model to enhance predictive performance. The LSTM network consists of three main LSTM layers, each with 100 units in the first two layers and 50 units in the third. Each LSTM layer is followed by a Dropout layer with a rate of 0.3 to prevent overfitting. The final output layer is a Dense layer with a single unit, making it suitable for regression tasks. The LSTM model captures temporal dependencies in the data, while the Dropout layers add regularization. After training, the LSTM model's predictions are combined with those from an XGBoost model, which is trained on the same features but operates independently. The combined predictions, obtained by averaging the outputs of both models, leverage the temporal strengths of LSTM and the predictive power of XGBoost, resulting in improved evaluation metrics. This ensemble approach ensures that the complementary strengths of both models are utilized for better traffic situation forecasting.

```python
xgb_model = xgb.XGBRegressor()  # Initialize the XGBoost model
xgb_model.fit(X_train.squeeze(), y_train)  # Train the XGBoost model on the actual labels

# Prediction on test data with the XGBoost model
predictions_xgb = xgb_model.predict(X_test.squeeze())
```

FIGURE 3.16 – Code of our combined model training and prediction

3.2.4.3 Model Testing

After training. The test data undergoes the same preprocessing steps as the training data, including scaling the features with the same MinMaxScaler and reshaping to match the input shape required by the LSTM model. The LSTM model then generates predictions for the test dataset, capturing the temporal dependencies it learned during training. Simultaneously, the XGBoost model, which was also trained on the same features but operates independently, makes its own predictions. To enhance predictive accuracy, the outputs from both models are averaged, combining the temporal strengths of the LSTM with the predictive power of XGBoost.

A line plot to compare the real and predicted traffic situations by our combined model :

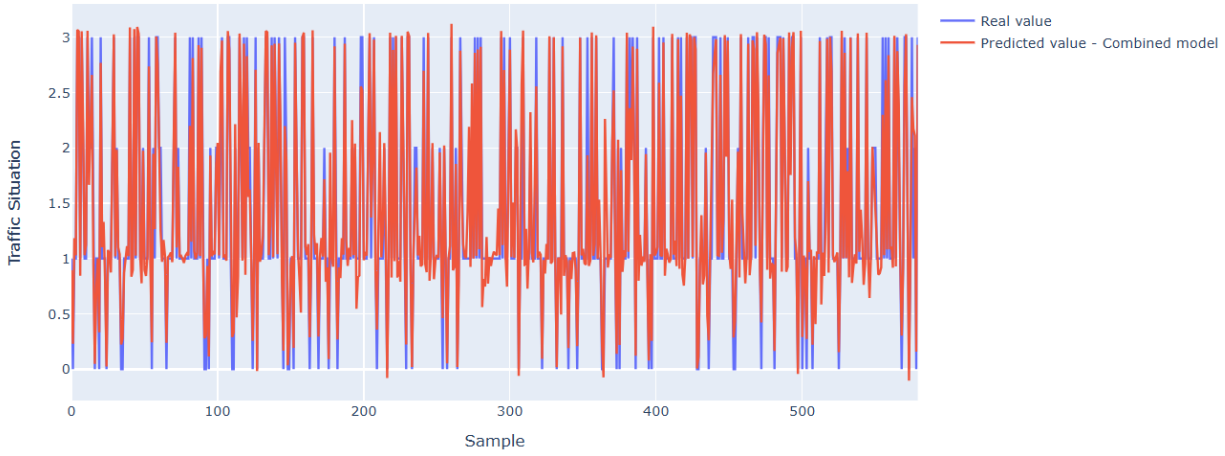


FIGURE 3.17 – Combined Model : Comparison of Real and Predicted Values

Here, a comparison between the actual traffic situation values and the predicted values generated by the combined LSTM and XGBoost model after averaging results.

	real values	Predicted Values
155	1.0	1.068664
132	2.0	1.932298
297	1.0	1.027954
197	1.0	0.844515
34	0.0	0.227470
490	1.0	0.937688
94	0.0	0.150836
58	3.0	2.998081
263	2.0	1.864565
227	2.0	2.031373

FIGURE 3.18 – Combined Model : Real Values and Predicted Values

3.2.4.4 Model Evaluation

Evaluation metrics include MSE, MAE, and RMSE (Root Mean Squared Error) calculated on the test dataset to assess prediction accuracy. Visualizations such as line plots and bar charts illustrate the comparison between actual and predicted traffic situations, highlighting the effectiveness of the combined LSTM and XGBoost approach in traffic prediction tasks.

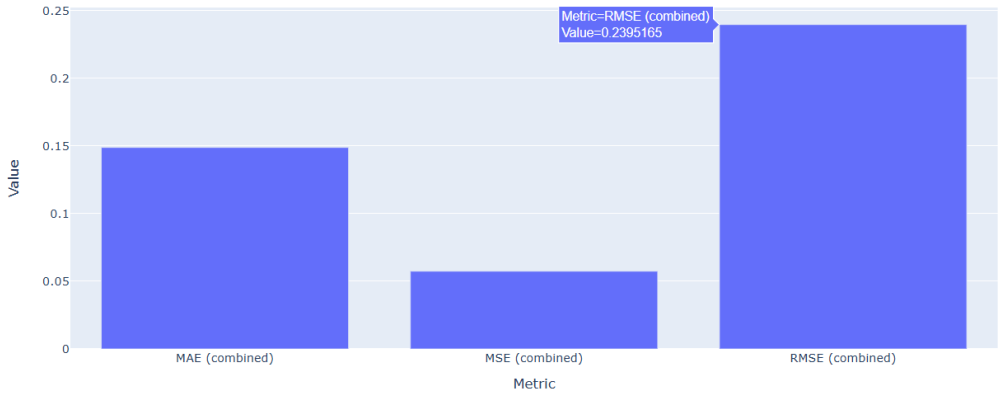


FIGURE 3.19 – Combined Model : Model Evaluation based on RMSE, MAE, MSE values

Moving to the Combined LSTM + XGBoost model outperformed the Basic LSTM and Improved LSTM models by achieving significantly lower RMSE, MAE, and MSE values. This improvement highlights the synergistic benefits of integrating LSTM for capturing temporal patterns and XGBoost for handling nonlinear relationships in traffic data. The combined approach resulted in more accurate predictions, making it a robust solution for enhancing short-term traffic forecasting accuracy.

3.2.5 Conclusion

The comparison of three LSTM models demonstrates a clear progression towards enhanced predictive accuracy in traffic situation forecasting.

Model	**RMSE**	**MAE**	**MSE**
Basic LSTM	0.397	0.31	0.15
Improved LSTM	0.316	0.22	0.1
Combined LSTM + XGBoost	0.23	0.14	0.05

TABLE 3.2 – Comparison of LSTM Models Performance

This integrated approach not only outperformed standalone LSTM variants but also capitalized on LSTM's ability to capture temporal dependencies and XGBoost's strengths in handling complex relationships and improving model stability through ensemble learning.

These results underscore the efficacy of combining Results of LSTM with those made by XGBoost for achieving superior predictive performance in traffic forecasting applications, offering more accurate insights crucial for effective decision-making in urban transportation management and planning.

Chapter 4

Deployement

In this chapter, we will explore the implementation of a system that prioritizes vehicles equipped with cameras by simulating a traffic control mechanism using Arduino. This involves the detection of camera-equipped vehicles and dynamically adjusting traffic lights to green, ensuring a smoother and more efficient traffic flow for these priority vehicles. We will begin by discussing the necessary hardware components, followed by the software implementation and coding. Finally, we will present and analyze the results of our simulation. This project aims to demonstrate how intelligent traffic systems can be enhanced through the integration of modern technology to improve urban mobility and safety.

4.1 Hardware Components

Detailed Overview of the Arduino-Based Traffic Light Simulation

In our Arduino-based traffic light simulation, we integrated various components to ensure precise functionality and control :

— **Arduino Uno Microcontroller** : Acts as the central controller, responsible for managing the sequencing and timing of the traffic light LEDs. It processes input signals from connected devices to dynamically adjust traffic light states.

FIGURE 4.1 – Arduino Uno

— **USB Cable** : Links the Arduino Uno to a computer, providing both power and a communication interface for programming and data exchange. This connection is essential for real-time adjustments and updates to the traffic light system.

FIGURE 4.2 – USB Cable for Arduino UNO

— **Jumper Wires** : Serve as flexible connectors between the Arduino Uno, breadboards, and other components. These wires enable easy prototyping and circuit adjustments without the need for soldering, facilitating rapid development and testing.

FIGURE 4.3 – Jumper Wires

— **Resistors** : Play a crucial role in the circuit by limiting current flow through the traffic light LEDs. This protection ensures the longevity and proper operation of the LEDs, preventing them from burning out due to excessive current.

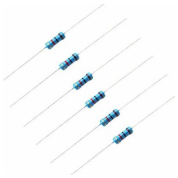


FIGURE 4.4 – Resistors

— **LEDs** : Represent the traffic light signals, with different colors (red, yellow, green) indicating various states of the traffic flow. LEDs provide clear visual feedback on the current traffic light sequence managed by the Arduino Uno.

FIGURE 4.5 – LEDs

— **Breadboards** : Act as the platform for assembling and testing the circuitry. They allow for the temporary connection of components and facilitate quick modifications during the development process. Breadboards are instrumental in prototyping before finalizing the circuit design.

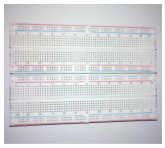


FIGURE 4.6 – Breadboards

— **Webcam Integration** : A webcam connected to a PC captures continuous video footage, which is processed using the YOLO (You Only Look Once) object detection algorithm. This system identifies emergency vehicles within the video stream based on predefined characteristics. Once an emergency vehicle is detected, the information is communicated to the Arduino Uno via serial communication protocols.

FIGURE 4.7 – Webcam

— **PC Internal Microphone Integration** : The PC's internal microphone monitors ambient audio for specific sound patterns associated with emergency vehicle sirens. Real-time audio analysis software processes these signals to detect sirens. Upon detection, the PC sends a signal to the Arduino Uno, prompting it to adjust the traffic lights to green, prioritizing the passage of emergency vehicles.

— **Small-Scale Replica of a Moroccan Police Car** : Integrating a small-scale replica of a Moroccan police car visually demonstrates the system's capability to detect and respond to emergency vehicles. This physical model enhances the simulation by providing a tangible representation of how the traffic lights adjust in real-time, showcasing the practical application of the integrated detection and control system.

FIGURE 4.8 – Small-Scale Replica of a Moroccan Police Car

This comprehensive setup illustrates the synergy between hardware components, real-time data processing, and microcontroller programming in enhancing traffic management systems through intelligent control mechanisms.

4.2 Software Implementation

The implemented Python script integrates OpenCV for video capture and object detection using the YOLOv8 model, specifically trained to identify ambulance, fire truck, and police vehicles. It utilizes a separate thread for real-time audio detection through PyAudio, monitoring for emergency vehicle sirens. Upon detecting an emergency vehicle both visually and audibly, it sends a signal to an Arduino Uno via serial communication to adjust traffic lights, prioritizing emergency vehicle passage. In the implemented Python script, two main detection mechanisms are employed : visual detection using YOLOv8 and audio detection using PyAudio.

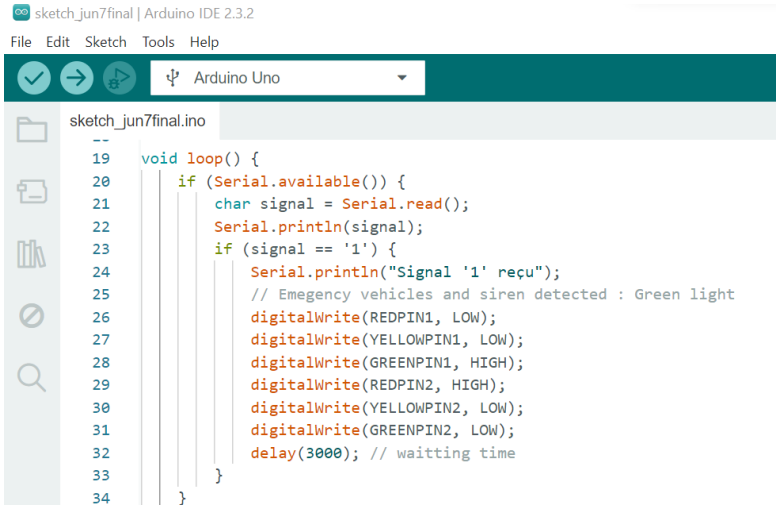
Integration and Response

The system integrates both visual and audio detection mechanisms to enhance traffic management. The following Python script demonstrates the integration of real-time visual and audio detection for emergency vehicles using OpenCV, YOLOv8, PyAudio, and Arduino. The script captures video from a camera, detects emergency vehicles visually with YOLOv8, and monitors for sirens using audio detection. When both visual and audio detections confirm the presence of an emergency vehicle with a siren, a signal (`b'1'`) is sent to an Arduino Uno via serial communication (`arduino.write(b'1')`). This signal instructs the Arduino to adjust traffic lights to prioritize the passage of the emergency vehicle. If no emergency vehicle with a siren is detected, a signal (`b'0'`) is sent (`arduino.write(b'0')`) to maintain normal traffic flow.

```python
def update_frame():
    global audio_detected
    ret, frame = cap.read()
    frame = cv2.resize(frame, (640, 480))
    result = model(frame, stream=True)
    visual_detected = False

    for info in result:
        boxes = info.boxes
        for box in boxes:
            confidence = box.conf[0]
            confidence = math.ceil(confidence * 100)
            Class = int(box.cls[0])
            if confidence > 60:
                x1, y1, x2, y2 = box.xyxy[0]
                x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2)
                cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 0, 255), 5)
                cvzone.putTextRect(frame, f'{classnames[Class]} {confidence}%', [x1 + 8, y1 + 100], scale=1.5, thickness=2)
                if Class in [0, 1, 2]:
                    print("Emergency vehicle detected  !")
                    visual_detected = True

    print(f"audio_detected: {audio_detected}, visual_detected: {visual_detected}")

    if visual_detected and audio_detected:
        print("Emergency vehicle with siren detected ! Sending the signal to the Arduino...")
        arduino.write(b'1')
        time.sleep(0.1)
        visual_detected = False
        audio_detected = False
    else:
        arduino.write(b'0')

    cv2image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGBA)
    img = Image.fromarray(cv2image)
    imgtk = ImageTk.PhotoImage(image=img)
    label.imgtk = imgtk
    label.configure(image=imgtk)
    label.after(10, update_frame)
```

Arduino Uno Control Logic for Traffic Light Adjustment

The void loop() function in our Arduino implementation serves as the central control mechanism for dynamically adjusting traffic lights based on real-time signals received via serial communication. This function continuously monitors incoming data from a connected computer, typically generated by a Python script that detects emergency vehicles using visual and audio cues. Upon receiving a specific signal (typically '1'), indicating the presence of an emergency vehicle with sirens detected, Arduino executes commands to switch the traffic lights. It transitions the relevant lights to green, allowing priority passage for emergency vehicles while halting cross traffic momentarily for safe traversal.



Here you can Find the Software Implementation codes : Python script that detects emergency vehicles using visual and audio cues and Arduino Uno Control Logic Code : Link of Software Implementation

4.3 Practical Experimentation

Step 1 : Using all the integrated materials, and YOLO object detection algorithm the system effectively prioritizes emergency vehicles during critical situations. When an emergency vehicle, identified by sirens and visual cues, approaches, the Arduino Uno swiftly adjusts the traffic lights to grant it a clear path.

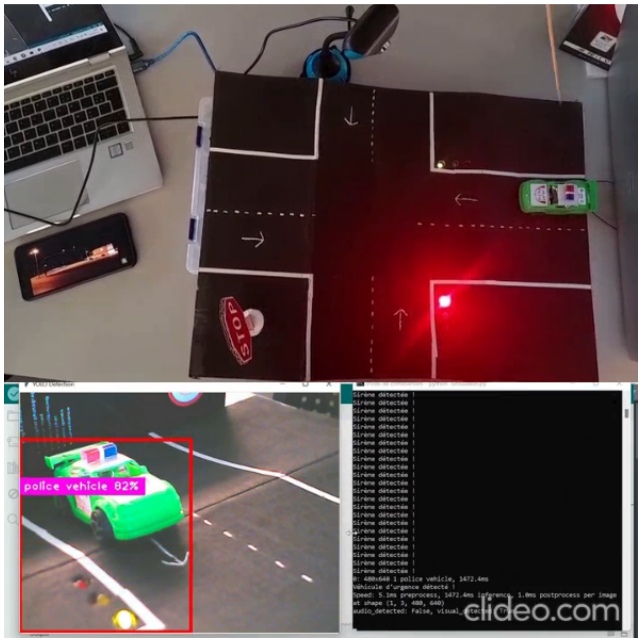


FIGURE 4.9 – Emergency Vehicle Detection and Priority Response

Step 2 : Once the emergency vehicle passes through the intersection and the siren stops, the system

verifies the road's safety before proceeding.

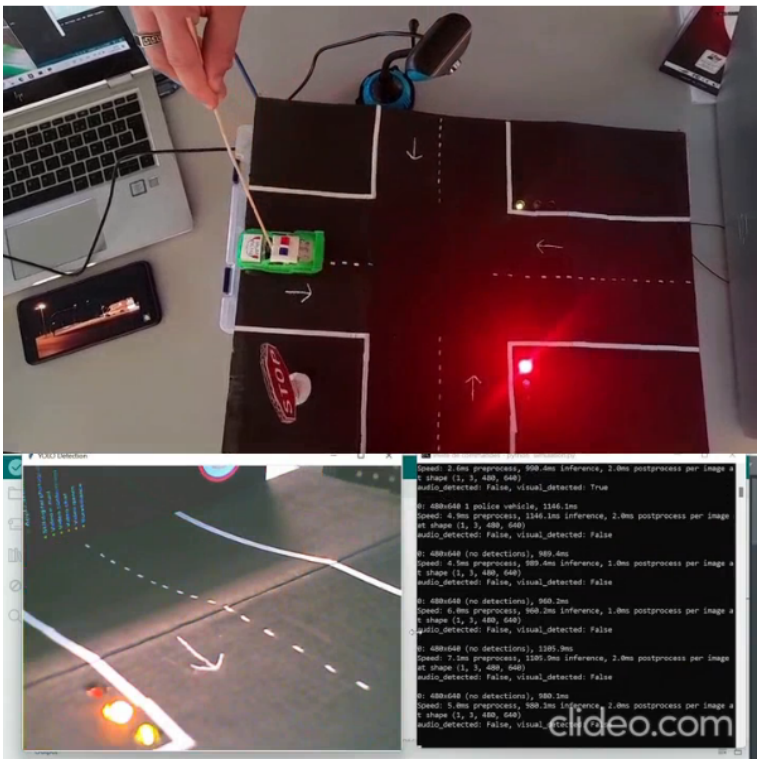


FIGURE 4.10 – Road Clearance Verification

Step 3 : After confirming that the emergency vehicle has passed and there is no siren , the traffic light returns to its regular mode to resume normal traffic operations.

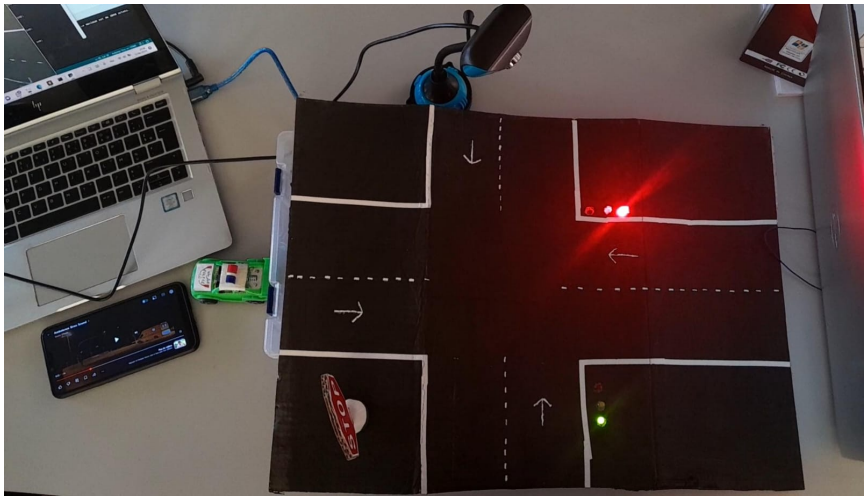


FIGURE 4.11 – Transition to Regular Traffic Lights Mode

Here You can Find the Video of the Simulation in Real-Time : Link of Real-Time Simulation

4.4 Conclusion

In summary, we have successfully developed and implemented an intelligent traffic management system that seamlessly integrates hardware components with advanced software algorithms. Our Arduino-based traffic light controller, coupled with visual and audio detection mechanisms, effectively prioritizes emergency vehicles in real-time traffic scenarios. The system's ability to swiftly adjust traffic signals upon detecting emergency

vehicles, and then revert to normal operations once the situation is resolved, demonstrates its practical utility in enhancing both emergency response times and overall traffic flow efficiency.

Chapter 5

Conclusion

5.1 Summary of the Project's Objectives and Accomplishments

This project successfully developed an intelligent traffic management system that prioritizes emergency vehicles using the YOLOv8 object detection algorithm and predicts traffic congestion through LSTM and XGBoost models. The results demonstrated that the system has the potential to significantly improve emergency response times and overall traffic flow. Despite the challenges encountered, the integration of machine learning and hardware components proved effective in creating a robust and reliable solution.

The future work will focus on addressing the identified problems, integrating more real-time data sources, and expanding the system's capabilities to handle a broader range of scenarios. Overall, this project lays a solid foundation for the development of advanced intelligent traffic management systems, promising substantial benefits for urban mobility and public safety.

5.2 Difficulties Limitations

Throughout the development and implementation of this project, several challenges and limitations were encountered. The inconsistency in data quality affected the accuracy of both the vehicle detection and traffic prediction models. Synchronizing different hardware components, such as Arduino boards and cameras, presented difficulties that needed careful management to ensure reliable operation. Real-world testing revealed occasional delays in the system's response time , which could potentially impact its effectiveness in critical situations. Additionally, there were challenges related to the scarcity of Moroccan-specific data and external sources that could assist in our research. Furthermore, both the internal and external cameras exhibited slow detection speeds and the low quality and the positioning of our camera for a perfect view, which sometimes results in very approximate detection

5.3 Future Work

In the realm of future enhancements, our intelligent traffic management system will focus on integrating real-time data sources such as live traffic feeds and emergency dispatch information. This integration aims to bolster the system's agility and accuracy in responding to dynamic traffic conditions and emergency incidents promptly. Expanding the system's capabilities beyond its current focus on emergency vehicle detection and traffic congestion prediction presents an opportunity to handle a broader range of emergency scenarios, including natural disasters and public events. Additionally, exploring advanced machine learning techniques such as reinforcement learning holds promise for refining traffic prediction models, enabling the system to adapt and optimize traffic flow in real-time based on evolving environmental and operational conditions. Establishing a comprehensive simulation environment will be instrumental in testing various system configurations and scenarios, ensuring robust performance and facilitating iterative improvements. Deploying the system in larger urban environments will provide critical insights into scalability and effectiveness, guiding further enhancements and optimizations tailored to diverse urban settings.Lastly, incorporating future versions of YOLO for enhanced object detection capabilities promises to further elevate the system's precision and efficiency in vehicle detection within complex traffic environments.

5.4 General Conclusion

This project represents a significant step forward in the development of intelligent traffic management systems, addressing two critical aspects of urban mobility : emergency vehicle detection and traffic congestion prediction. By leveraging cutting-edge technologies and innovative approaches, we have created a comprehensive solution that has the potential to transform urban traffic management and emergency response. Our work on emergency vehicle detection, utilizing the advanced YOLOv8 algorithm, demonstrates the power of modern object detection techniques in real-world applications. The ability to accurately and swiftly identify ambulances, police cars, and fire trucks in complex traffic scenarios is crucial for ensuring rapid response times and potentially saving lives. This system not only enhances road safety but also contributes to the overall efficiency of emergency services in urban environments.

The traffic congestion prediction component of our project, combining LSTM networks with XGBoost, showcases the potential of hybrid machine learning approaches in tackling complex, time-dependent problems. By accurately forecasting traffic conditions, this system enables proactive traffic management, potentially reducing commute times and improving the overall quality of urban life. The integration of this predictive capability with the emergency vehicle detection system creates a synergistic solution that can dynamically adjust traffic flow to prioritize emergency responses while managing general congestion.

Furthermore, the incorporation of physical hardware components, such as Arduino boards, cameras, and sound detectors, bridges the gap between theoretical models and practical implementation. This integration demonstrates the feasibility of deploying such advanced systems in real-world settings, paving the way for future smart city initiatives. In the broader context of urban development and smart city technologies, our project aligns with global efforts to create more efficient, safer, and more livable urban environments. As cities worldwide grapple with increasing population densities and the challenges of modernizing infrastructure, solutions like ours offer a glimpse into the future of urban mobility management.

While our project has achieved significant milestones, it also opens up numerous avenues for future research and development. The potential for integrating real-time data sources, expanding to additional emergency scenarios, and exploring more advanced machine learning techniques suggests that this field of study remains rich with possibilities. In conclusion, our work not only contributes to the academic understanding of traffic management and emergency response systems but also offers practical solutions to real-world challenges. As we look to the future, the continued development and refinement of such systems will play a crucial role in shaping smarter, safer, and more efficient cities for generations to come.

Bibliography

[1] Les réseaux de neurones récurrents | Reccurent neural network - Deep learning - - Kongakura. https ://kongakura.fr/article/Les-reseaux-de-neurones-reccurent.

[2] Bassant M. Elbagoury, Luige Vladareanu, Victor Vlădăreanu, Abdel Badeeh Salem, Ana-Maria Travediu, and Mohamed Ismail Roushdy. A Hybrid Stacked CNN and Residual Feedback GMDH-LSTM Deep Learning Model for Stroke Prediction Applied on Mobile AI Smart Hospital Platform. *Sensors*, 23(7) :3500, January 2023.

[3] A Transfer-Learning-Based Approach for Emergency Vehicle Detection. *Eurasian Journal of Science and Engineering*, 8(1), 2022.

[4] Road Safety in Morocco | Traffic accidents, crash, fatalities & injury statistics | GRSF. https ://www.globalroadsafetyfacility.org/country/morocco.

[5] Kaabeche Oussama. EXTRACTION DES ILOTS PAR DEEP LEARNING.

[6] What is YOLOv8 ? The Ultimate Guide. [2024]. https ://blog.roboflow.com/whats-new-in-yolov8/.

[7] LSTM network : A deep learning approach for short-term traffic forecast - Zhao - 2017 - IET Intelligent Transport Systems - Wiley Online Library. https ://ietresearch.onlinelibrary.wiley.com/doi/full/10.1049/iet-its.2016.0208.